Scenario 1: Greenfield, Build a URL Shortening Service

Requirement

"Build a URL shortening service."

Requirement Understanding

At face value this is a simple CRUD application, but building it correctly requires clarity on several dimensions.

Clarifying questions asked (and assumed answers for this assessment):

1. Does the short code need to be globally unique forever, or only while active? Forever unique.
2. Should we support custom aliases? Yes, enterprise users want branded links.
3. Should links expire? Yes, with optional TTL per link.
4. Is click tracking required? Yes, basic analytics per link.
5. What is the expected read:write ratio? Assumed 100:1, redirects are far more frequent than creations.
6. Single-tenant or multi-tenant? Single-tenant prototype.
7. Rate limiting requirements? Protect the creation endpoint from spam.

Task Decomposition

Phase 1: Core data layer. `ShortUrl` entity (code, original URL, alias, expiry, IP fields); `ShortUrlRepository` with `findByCode`, `existsByCustomAlias`, and an expiry query; PostgreSQL datasource with Hibernate auto-DDL.

Phase 2: Short code generation. Counter-based Base62 generator using Redis `INCR`; batch allocation (1000 IDs per Redis call); thread-safety via `synchronized` on the generate method. Dependency: Redis has to be available first.

Phase 3: Write path. `WriteService.shorten()` validates alias uniqueness, generates the code, saves, and caches it. `WriteService.delete()` removes from the DB and evicts the Redis key. `ShortenRequest`/`ShortenResponse` DTOs. Dependency: Phases 1 and 2 complete.

Phase 4: Read path (redirect). `ReadService.resolveAndTrack()` does a cache-first lookup with a DB fallback; `RedirectController` returns HTTP 302; `UrlNotFoundException` (404) and `UrlExpiredException` (410) handle the failure cases. Dependency: Phase 1 and Redis config complete.

Phase 5: API layer. `UrlController` (POST, GET list, GET by code, DELETE); `GlobalExceptionHandler`; CORS config; rate limiting on POST.

Phase 6: Testing. Unit tests for `ShortCodeGenerator` (uniqueness, Base62 format, non-null), `WriteService` (happy path, custom alias, duplicate alias), `ReadService` (cache hit, cache miss, not found, expired); integration tests for `UrlController` (201, 400, 409, list).

Implementation Approach

`ShortCodeGenerator`**.** The first design question was hashing, MD5/SHA plus truncation, versus a counter. I went with a counter: no collision probability, shorter codes for the same uniqueness guarantee, and Base62-encoded increments are URL-safe by construction. Batch size 1000 balances Redis call frequency against wasted IDs on restart.

`WriteService`**.** Deliberately simple, no transaction management beyond default JPA behavior. The custom-alias check-then-save has a theoretical TOCTOU race, but the database UNIQUE constraint is the real safety net here, and the application-level check exists purely to give a friendlier error message before the constraint would reject it anyway.

`ReadService`**.** Cache-aside: Redis first, PostgreSQL fallback, repopulate Redis. The 24h Redis TTL means an expired URL can still resolve from cache for up to 24h after expiry if it was cached. I accepted this because expiry is enforced on the DB path, and cache-served URLs were already validated when cached. A tighter solution would set `TTL = min(cache_ttl, time_until_expiry)`.

302 vs. 301. 302 was chosen; see `ARCHITECTURE.md` for the reasoning.

Acceptance Criteria

Criterion	Verified by
POST with valid URL returns 201 and a short URL	`UrlControllerIT.shortensUrlAndReturns201`
POST with invalid URL returns 400	`UrlControllerIT.returns400ForInvalidUrl`
POST with duplicate alias returns 409	`UrlControllerIT.returns409ForDuplicateAlias`
GET `/{code}` redirects with 302	Manual curl test
GET `/{code}` for expired URL returns 410	`ReadServiceTest.throwsExpiredForExpiredUrl`
GET `/{code}` for unknown code returns 404	`ReadServiceTest.throwsNotFoundForUnknownCode`
GET list returns array	`UrlControllerIT.listAllReturnsUrls`
DELETE removes URL and Redis key	`WriteServiceTest`
Generated codes unique within a batch	`ShortCodeGeneratorTest.generatesUniqueCodesWithinBatch`
Generated codes are alphanumeric	`ShortCodeGeneratorTest.generatedCodeIsBase62`

Validation Approach

1. Unit tests run in isolation with Mockito mocks, no infrastructure required.
2. Integration tests use H2 (`application-test.yml`) and mock Redis/Kafka via `@MockBean` where needed. These are now actually executing via `mvn verify` (see `AI_WORKFLOW.md` for the Failsafe wiring fix).
3. End-to-end validation via Docker Compose with real PostgreSQL, Redis, and Kafka.
4. Manual API testing with curl against the running stack.

AI Generation Notes

All Java source files were generated by AI from a specification. I manually reviewed `ShortCodeGenerator.toBase62()` to verify the Base62 encoding algorithm. `ReadService.resolveAndTrack()` had the Kafka publish call added after review, it was missing from the first draft. The `@PrePersist` method on `ShortUrl` was verified to correctly set `createdAt` before insert. `application-test.yml` was manually adjusted: the H2 dialect and the `:test` counter-key suffix were added to avoid contaminating production Redis keys.